第14课 二维数组

一、课程信息

项目	内容
课次	第 14 课 / 共 40 课
主题	二维数组
前置知识	一维数组的定义与遍历（第10-11课）、函数入门（第13课）
后续衔接	第15课 枚举法（一）

教学目标（可检验）

1. **能正确声明并使用二维数组**：学生能独立写出 `int a[M][N];` 的声明，并通过 `a[i][j]` 准确读写指定位置的元素。
2. **能用双重循环遍历二维数组**：学生能编写嵌套 `for` 循环完成二维数组的输入和格式化输出。
3. **能用二维数组解决简单问题**：学生能完成"按格式输出矩阵""求每行之和/每列之和"等基础题目。

二、教学重点与难点

教学重点

- 二维数组的概念理解（从一维到二维的思维跃迁）
- 二维数组的声明语法和下标访问 `a[行][列]`
- 双重循环遍历二维数组的固定写法

教学难点

- **下标从 0 开始**的记忆与习惯养成（行列都从 0 开始）
- **行和列的对应关系**：第一个下标是行、第二个下标是列，不要写反
- **双重循环的嵌套逻辑**：外层控制行、内层控制列，理解执行顺序

三、课堂时间分配（共 100 分钟）

时间	时长	环节	内容
0′-5′	5 min	复习导入	回顾一维数组，引出"一排柜子不够用了"的问题
5′-25′	20 min	讲解一：概念与声明	二维数组的比喻、声明语法、下标规则
25′-35′	10 min	练习一：小试牛刀	在纸上写出声明、手动填表格
35′-55′	20 min	讲解二：双重循环遍历	例题1-例题2，输入输出二维数组
55′-70′	15 min	练习二：上机编码	课堂练习题1-2
70′-85′	15 min	讲解三：简单应用	例题3（求行和列和），讲解思路
85′-95′	10 min	练习三：综合实战	课堂练习题3-4
95′-100′	5 min	总结与作业	常见错误回顾、布置课后作业、预告下节课

> 说明：三段讲解分别为 20 分钟、20 分钟、15 分钟，均不超过 25 分钟，穿插练习保持注意力。

四、讲稿提纲

4.1 复习导入（5 分钟）

> 同学们，还记得我们学过的数组吗？我们说一维数组就像教室后面**一排储物柜**，每个柜子有一个编号，从 0 号开始。我们可以用 `a[0]`、`a[1]`、`a[2]` 来找到对应的柜子。
> 那老师问大家一个问题：如果学校要存放全年级每个班每个同学的成绩，只有一排柜子够吗？

（等待学生回答"不够"）

> 对！一排不够，我们需要**很多排**——就像一面大柜墙！今天我们就来学习这面"柜墙"——**二维数组**。

4.2 讲解一：概念与声明（20 分钟）

用生活比喻理解二维数组

> 想象一下，你们的教室座位就是一个二维数组！
> - 教室有**5排**座位，每排有**6个**位置。
> - 要找到一个同学坐在哪里，你需要说"**第几排、第几列**"——比如"第2排第3列"。
> - 一维数组只需要一个编号，二维数组需要**两个编号**：一个表示**行**（第几排），一个表示**列**（第几个位置）。

在黑板上画出表格：

```
        第0列  第1列  第2列  第3列
第0行  [    ] [    ] [    ] [    ]
第1行  [    ] [    ] [    ] [    ]
第2行  [    ] [    ] [    ] [    ]
```

> 这就像一个棋盘、一张电子表格、一面柜墙——横着看是**行**，竖着看是**列**。

声明语法

> 在 C++ 里，我们这样声明一个二维数组：

```
int a[3][4];  // 3行4列的整数表格
```

> 还记得一维数组 `int a[5];` 表示一排 5 个柜子吗？
> 现在 `int a[3][4];` 就是 **3 排，每排 4 个柜子**，一共 3×4 = 12 个格子！
> 第一个方括号里的数字是**行数**，第二个方括号里的数字是**列数**。

下标访问

> 要往某个格子里放东西或者取东西，用两个下标：

```
a[1][2] = 100;  // 第1行第2列的格子，放入100
cout << a[1][2]; // 把第1行第2列格子里的数拿出来看看
```

> **特别注意**：行和列都是从 **0** 开始的！就像一维数组的柜子从 0 号开始一样。
> 所以 `a[3][4]` 这个数组：
> - 行号是 0、1、2（共 3 行）
> - 列号是 0、1、2、3（共 4 列）
> - `a[3][0]` 就已经越界了哦！行号最大只能到 2。

初始化

> 我们也可以在声明的时候就把数据填好：

```
int a[2][3] = {
    {1, 2, 3},    // 第0行
    {4, 5, 6}     // 第1行
};
```

> 就像提前把每个座位上放好了名字牌。

4.3 练习一：纸上热身（10 分钟）

> 请同学们在纸上完成：
> 1. 画出 `int b[2][5];` 对应的表格，标出所有下标。
> 2. 如果 `b[0][3] = 7; b[1][1] = 9;`，把这两个数填进表格里。

（老师巡视，个别辅导，发现"行列写反"的同学重点提醒。）

4.4 讲解二：双重循环遍历（20 分钟）

> 一维数组我们用**一个 for 循环**来遍历。二维数组有行和列两个方向，所以我们需要**两个 for 循环嵌套**——还记得第 9 课学的循环嵌套吗？
> 外面那层循环管**行号**，里面那层循环管**列号**。就像你在教室里点名：先喊"第 0 排"，然后从第 0 列喊到最后一列；再喊"第 1 排"，又从第 0 列喊到最后一列……

然后带入例题1和例题2，边讲边写代码。

4.5 讲解三：简单应用（15 分钟）

> 学会了输入输出，我们来做点有用的事！
> 想象一下，你是班长，老师给你一张成绩表，每一行是一个同学的各科成绩。老师让你算出每个同学的总分，还要算出每一科的平均分。这就是"求行和"和"求列和"！

带入例题3，边讲边写代码。

4.6 总结口诀

> 最后老师送大家一句口诀：
> **"二维数组像表格，先行后列要记清；外循环管行号，内循环管列号；下标都从零开始，别忘减一不越界！"**

七、常见错误与应对

错误 1：行列下标写反

```
//  错误：想访问第2行第0列，却写成了第0行第2列
cout << a[0][2]; // 实际想写 a[2][0]
```

应对：反复强调口诀——**"先行后列"**。在黑板画表格时让学生指着说"第几行第几列"，形成肌肉记忆。

错误 2：下标越界

```
int a[3][4];
a[3][0] = 5; //  错误！行号最大只能到2
a[0][4] = 5; //  错误！列号最大只能到3
```

应对：带学生数一遍——"声明 `[3]` 就是 0、1、2 这三个，最大是 2，不是 3！"类比生活："教室有 3 排，编号是第 0 排、第 1 排、第 2 排，没有第 3 排哦。"

错误 3：双重循环的边界搞混

```
//  错误：内外层循环的上界用反了
for (int i = 0; i < m; i++) {      // 应该是 n（行数）
    for (int j = 0; j < n; j++) { // 应该是 m（列数）
        cin >> a[i][j];
    }
}
```

应对：强调"外层管行用 n，内层管列用 m"。建议学生**先写注释再写代码**：

```
// 外层：遍历每一行（共n行）
for (int i = 0; i < n; i++) {
    // 内层：遍历这一行的每一列（共m列）
    for (int j = 0; j < m; j++) {
```

错误 4：忘记在每行结束时换行

```
//  错误：所有数字输出在一行上
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        cout << a[i][j] << " ";
    }
    // 忘记写 cout << endl;
}
```

应对：提醒学生"内层循环结束 = 一行输出完毕 = 该换行了"。可以用教室点名比喻："喊完第 0 排所有同学后，要说一声'换下一排！'"

错误 5：数组大小开得太小

```
int a[3][3];
// 但题目输入的是 5×5 的矩阵  越界！
```

应对：教给学生好习惯——**数组大小按题目的最大范围来开**，宁可多开不能少开。告诉学生"柜子多了空着不要紧，柜子少了东西放不下就出大问题"。

九、下节课预告

> 同学们，下节课我们要学习一种非常有趣的解题方法——**枚举法**！
>
> 什么是枚举法呢？简单来说就是"**把所有可能的答案都试一遍，看看哪个是对的**"。
>
> 听起来是不是很"笨"？但其实很多时候，这是最靠谱的方法！就像你丢了一把钥匙，最靠谱的方法就是把每个角落都找一遍。
>
> 枚举法会用到我们学过的循环和判断，再加上今天学的数组，你们已经准备好啦！下节课见！